BHAC+ Developer and Physics Manual

Workspace documentation for the active compiled source tree

Snapshot documented from the local source tree on 2026-04-20

Contents

1 Purpose, Scope, and Reading Strategy **3**
- 1.1 What is meant by BHAC+ here . . . 3
- 1.2 Conventions used throughout this manual . . . 3

2 Build System, Runtime Usage, and Configuration **3**
- 2.1 Work directory model and setup script . . . 3
- 2.2 Compile-time controls . . . 4
- 2.3 Runtime namelists and user-facing parameters . . . 5
- 2.4 Important consistency rules . . . 6

3 Source Tree and Active Code Structure **6**

4 Global Control Flow **7**
- 4.1 Program entry and startup . . . 7
- 4.2 The time integration loop . . . 7
- 4.3 Advance and update splitting . . . 7

5 Variable Layout, State Vectors, and Preprocessor Notation **8**
- 5.1 Variable classes . . . 8
- 5.2 Hydrodynamic and spacetime variables . . . 8
- 5.3 Radiation variables . . . 9
- 5.4 Internal M1 state conventions . . . 9

6 Geometry, Coordinates, and Metric Infrastructure **9**
- 6.1 Physics and numerical role . . . 9
- 6.2 Core geometry routines in geometry.t . . . 9
- 6.3 Metric interface in mod_metric.t . . . 10
- 6.4 CFC metric solver . . . 10
- 6.5 GW backreaction . . . 11

7 EOS Framework **11**
- 7.1 Generic EOS interface in mod_eos.t . . . 11
- 7.2 Atmosphere model . . . 12
- 7.3 Tabulated EOS in mod_eos_tabulated.t . . . 12
- 7.4 Leptonic 4D EOS in mod_eos_leptonic.t . . . 13
- 7.5 Why the generic EOS helpers matter . . . 14

8 GRMHD Physics Layer **15**
- 8.1 Conserved and primitive variables . . . 15
- 8.2 Interface routines in amrvacphys.t . . . 15
- 8.3 Primitive recovery in mod_imhd_con2prim.t . . . 16
- 8.4 Intermediate variables in mod_imhd_intermediate.t . . . 16

8.5 Auxiliary GRMHD diagnostics in mod_physaux.t . . . 17

9 M1 Radiation Transport **17**
9.1 Physical model . . . 17
9.2 Species bookkeeping and startup . . . 18
9.3 Transport routines in mod_m1.t . . . 18
9.4 Closure and realizability in mod_m1_closure.t . . . 19
9.5 Implicit collisional source solve in mod_m1_collisional.t . . . 19
9.6 Rates and emissivity-absorption-scattering infrastructure in mod_m1_eas.t . . . 20
9.7 Weakhub gray tables . . . 21
9.8 Analytical rates . . . 21
9.9 Backreaction, beta equilibrium, diagnostics, and optical depth . . . 21

10 I/O, Conversion, Import, and Healpix Detectors **22**
10.1 Runtime I/O in amrio.t . . . 22
10.2 Conversion and post-processing in convert.t . . . 23
10.3 Analysis hooks . . . 23
10.4 External initial-data import . . . 23
10.5 Healpix outflow detectors . . . 24

11 Tests, Example Workflows, and Practical Use **24**
11.1 BNS test setups . . . 24
11.2 What the muonic test is demonstrating . . . 25
11.3 Why imported variable ordering matters . . . 25

12 Extension Guide and Common Pitfalls **25**
12.1 Adding a new EOS or modifying an existing one . . . 25
12.2 Adding new radiation species or rate models . . . 25
12.3 Common failure modes . . . 26

13 Utility Modules and Support Infrastructure **26**

14 Summary **26**

1 Purpose, Scope, and Reading Strategy

This document is a code-and-physics manual for the active BHAC+ source tree in this workspace. It is intended for three use cases:

1. understanding the global control flow from startup to time stepping, output, and shutdown;
2. locating the source file and routine that implements a given physical model or numerical step;
3. extending the code consistently, in particular for realistic EOS work, neutrino transport, initial-data import, and diagnostics.

1.1 What is meant by BHAC+ here

In this workspace, BHAC+ means the BHAC code path selected by the top-level makefiles:

- core driver files in `BHAC/src`;
- GRMHD physics in `BHAC/src/imhd`;
- geometry and metric modules in `BHAC/src/geometry` and `BHAC/src/geometry/cfc`;
- EOS modules in `BHAC/src/eos_modules`;
- M1 radiation transport in `BHAC/src/m1`;
- I/O, import, conversion, and detector infrastructure in `BHAC/src/amrvacio` and `BHAC/src/ID_reader`.

The repository also contains historical, backup, debug, and draft files such as dated variants, `*_old.t`, `*_debug.t`, and draft routines like `calculate_m1_eas_ixD_new*.t`. Those files are useful as development history, but they are *not* the reference implementation documented here unless explicitly stated. This manual describes the active compiled path that the main makefiles select.

1.2 Conventions used throughout this manual

- **State naming.** "Primitive" means variables such as ρ, T, Y_e, velocity, and magnetic field. "Conservative" means the evolved finite-volume or finite-difference state.
- **Indices.** The code uses the AMRVAC/BHAC preprocessor language. Common shorthand includes `ixI^L` for an input index box, `ixO^L` for an output box, `ix^D` for a single point loop index, `^C` for component loops, and `^KSP` for radiation-species loops.
- **Units.** The EOS and M1 modules mostly work in BHAC geometric/code units internally, with explicit conversions to or from cgs when reading tables or evaluating table-based microphysics.
- **Compile-time switches.** Much of the logic is enabled conditionally by macros such as `DY_SP`, `TABEOS`, `M1`, `M1_RATES`, `M1_TAU_OPT`, and `GW_BR`. The runtime namelists only work correctly if the corresponding compile-time path is present.

2 Build System, Runtime Usage, and Configuration

2.1 Work directory model and setup script

BHAC is designed to be built from a *work directory*. The script `BHAC/setup.pl` copies or edits a local `makefile`, `definitions.h`, and `amrvacsettings.t` in that work directory and points them back to the source tree through the environment variable `BHAC_DIR`.

The usual setup flow is:

```
export BHAC_DIR=/path/to/BHAC
mkdir run
cd run
cp /path/to/amrvacusr.t .
cp /path/to/amrvacusrpar.t .
perl $BHAC_DIR/setup.pl -d=33 -g=64,64,32 -p=imhd -coord=cartesian -arch=<arch>
make bhac
./bhac -i import.par
```

The most important `setup.pl` switches are:

- `-d=33`: set `ndim=3`, `ndir=3`;
- `-g=n1,n2,n3`: change `ixGhi^D` in `amrvacsettings.t`;
- `-p=imhd`: select the physics module;
- `-eos=<name>`: set the EOS selector in the work-directory makefile;
- `-coord=<name>`: choose the coordinate family;
- `-arch=<arch>`: choose compiler and library settings.

2.2 Compile-time controls

The main makefiles split the code into a core layer plus directory-specific layers:

- `BHAC/src/makefile`: core driver and AMR framework;
- `BHAC/src/imhd/makefile`: GRMHD physics;
- `BHAC/src/eos_modules/makefile`: EOS support;
- `BHAC/src/m1/makefile`: M1 radiation transport;
- `BHAC/src/geometry/makefile` and `BHAC/src/geometry/cfc/makefile`: coordinate and CFC/GW infrastructure;
- `BHAC/src/amrvacio/makefile`: I/O and conversion;
- `BHAC/src/modules/makefile`: generic utilities.

The most important compile-time switches and their meaning are listed in Table 1.

Table 1: Main compile-time switches relevant for the active BHAC+ workflow.

Switch	Location / effect	Meaning in the code path
`DY_SP`	geometry / GRMHD	Enables dynamical spacetime variables such as `alp_metric_`, `beta_metric^C_`, `psi_metric_`, together with the CFC-oriented hydrodynamic path.
`TABEOS`	variable registry / IMHD	Adds realistic-EOS composition variables `Dye_`, `Dymu_`, `ye_`, and `ymu_`. The active EOS helpers `eos_uses_ye` and `eos_has_ymu` key off this logic.

Switch	Location / effect	Meaning in the code path
`M1`	variable registry / transport	Adds radiation moments for each compiled species: `nrad^KSP_`, `erad^KSP_`, `frad^KSP^C_`.
`M1_RATES`	M1 microphysics	Activates emissivity, absorption, scattering, and number-rate infrastructure via `mod_m1_eas.t`.
`M1_TAU_OPT`	M1 microphysics	Adds optical-depth support and the extra source-array slot `tau_path`.
`GW_BR`	CFC / post-processing	Adds gravitational-wave backreaction variables and solvers.
`-nspecies=3`	preprocessor flag in makefile	Compile three radiation species, typically ν_e, $\bar{\nu}_e$, and ν_x.
`-nspecies=5`	preprocessor flag in makefile	Compile five radiation species, namely the three standard species plus ν_μ and $\bar{\nu}_\mu$. In the current active path this is the required build for muonic radiation transport.

2.3 Runtime namelists and user-facing parameters

The main runtime reader is `BHAC/src/amrvacio/amrio.t`. The most important namelists for daily use are:

- `filelist`: file names, conversion targets, primitive-name ordering for import;
- `savelist`: snapshot and slice cadence;
- `methodlist`: Riemann solver, limiter, update scheme, inversion method, and small-value handling;
- `boundlist`: boundary-condition strings;
- `amrlist`: AMR hierarchy and domain size;
- `cfclist`: conformal-flatness and metric-update controls;
- `eoslist`: EOS type and EOS table names;
- `m1list`: transport species, rate mode, floors, tables, and microphysical toggles;
- `outflowlist`: Healpix detector settings.

The current EOS and M1 combinations are:

1. **Plain baryonic realistic EOS:** `eos_type_input='tabulated'` with a 3D EOS table.
2. **Leptonic 4D EOS:** `eos_type_input='leptonic'` with both `baryon_table_name` and `leptonic_table_nam`
3. **Three-spec M1 with Weakhub:** `m1_use_neutrinos=T`, `m1_use_muons=F`, `m1_rates_Weakhub=T`.
4. **Three-spec M1 with analytical rates:** same species choice, but `m1_rates_Weakhub=F`, `m1_rates_analytical=T`.
5. **Five-spec muonic M1:** `m1_use_neutrinos=T`, `m1_use_muons=T`, `m1_use_5_species=T`, `m1_rates_Weakhub`
The active code explicitly forbids analytical five-spec mode.

2.4 Important consistency rules

The active code enforces several consistency checks at startup, and they should be treated as part of the user guide:

- five-spec transport requires the leptonic EOS path and `lep_use_muons=.true.`;
- muonic transport requires a matching muonic Weakhub table;
- the active gray Weakhub path currently supports `Weakhub_grey_method = 1`;
- build-time species count and runtime species choice must match;
- restart files generated with three species are not restart-compatible with five-spec builds because the state-vector layout differs.

3 Source Tree and Active Code Structure

Table 2: Directory-level structure of the active compiled source tree.

Directory	Main responsibility	Representative files
`BHAC/src`	top-level driver, AMR, global state, reconstruction, update driver	`amrvac.t`, `advance.t`, `finite_volume.t`, `finite_difference.t`, `mod_variables.t`
`BHAC/src/imhd`	GRMHD physics interface, primitive/conservative transforms, fluxes, sources	`amrvacphys.t`, `mod_imhd_con2prim.t`, `mod_imhd_intermediate.t`, `mod_physaux.t`
`BHAC/src/eos_modules`	generic EOS interface, tabulated EOS, realistic-EOS table readers	`mod_eos.t`, `mod_eos_tabulated.t`
`BHAC/src/eos_modules/leptonic`	1D leptonic EOS tables and baryon+electron+muon composition logic	`mod_eos_leptonic.t`, `mod_eos_leptonic_parameters.t`
`BHAC/src/geometry`	coordinate-system and metric helpers, geometric operators	`geometry.t`, `mod_metric.t`, `mod_coord_*.t`
`BHAC/src/geometry/cfc`	conformal-flatness metric solver and GW backreaction	`mod_cfc.t`, `mod_cfc_psi.t`, `mod_cfc_alp.t`, `mod_cfc_beta.t`, `mod_gw_br.t`
`BHAC/src/m1`	M1 transport, closure, implicit collisional solve, microphysics, diagnostics	`mod_m1.t`, `mod_m1_closure.t`, `mod_m1_collisional.t`, `mod_m1_eas.t`
`BHAC/src/amrvacio`	parameter input, snapshot I/O, conversion, analysis, Healpix detectors	`amrio.t`, `convert.t`, `analysis.t`, `mod_healpix_det.t`
`BHAC/src/ID_reader`	import of external initial data from XNS and FIL/Carpet-style HDF5 data	`mod_c2b.t`, `mod_c2b_cart3d.t`, `mod_xns.t`

Directory	Main responsibility	Representative files
`BHAC/src/modules`	generic utilities: constants, rootfinding, interpolation, timing, linear algebra	`mod_rootfinding.t`, `mod_interpolate.t`, `mod_lu.t`, `mod_lnsrch.t`
`BHAC/tests`	user problem setups, test makefiles, example parameter files	`BNS_SFHO`, `BNS_SFHO_muons`

4 Global Control Flow

4.1 Program entry and startup

The main executable entry point is the program `amrvac` in `BHAC/src/amrvac.t`. The startup sequence is:

1. start MPI communications through `comm_start`;
2. register variables with `setup_variables`;
3. read command-line and namelist parameters through `readcommandline` and `readparameters`;
4. initialize storage and communication types;
5. call `initglobal` to initialize physics globals;
6. if M1 is compiled, call `m1_startup`;
7. either read a restart snapshot or initialize level-one grids, then build the AMR tree;
8. initialize boundary-flux arrays, multigrid, metric solvers, import data, and user hooks;
9. enter the time-integration loop through `timeintegration`.

4.2 The time integration loop

The routine `timeintegration` in `amrvac.t` is the top-level evolution driver. Its logic is:

1. select active grids and compute the next time step via `setdt`;
2. run requested output through `saveamrfile`;
3. call `advance(it)` for one hydro/M1 update;
4. evaluate residuals, regrid if needed, and rebuild detectors if the AMR tree changed;
5. update counters and physical time;
6. terminate on `itmax`, `tmax`, residual bounds, or user interruption.

4.3 Advance and update splitting

The routine `advance` in `BHAC/src/advance.t` is the bridge between the global loop and the per-grid numerical update. Its responsibilities are:

- source splitting before transport when enabled;
- thermal conduction subcycling if compiled;

- implicit source updates through `addsource_impl` for stiff radiation coupling;
- dimensional splitting or unsplit transport through `advect`;
- post-transport source updates;
- primitive recovery after the update;
- metric updates through the CFC solver and optional GW backreaction;
- boundary-condition refresh for the next step.

The transport branch itself goes through `advect`, `advect1`, `process1_grid`, and `advect1_grid`. These routines coordinate:

- reconstruction;
- flux evaluation;
- geometric source terms;
- explicit and implicit source splitting;
- per-grid M1 helper-state allocation;
- calls into `finite_volume.t` or `finite_difference.t`.

5 Variable Layout, State Vectors, and Preprocessor Notation

5.1 Variable classes

The variable registry lives in `BHAC/src/mod_variables.t`. The routine `setup_variables` assigns the global indices used everywhere else. The code distinguishes:

- conservative variables (`cons_var`);
- primitive variables (`prim_var`);
- tracers (`tracer_var`);
- auxiliary variables (`aux_var`);
- extra variables (`extra_var`);
- GW-backreaction variables (`gwbr_var`);
- radiation variables (`radiation_var`);
- radiation implicit-source variables (`radiation_impl_var`).

5.2 Hydrodynamic and spacetime variables

In the active dynamical-spacetime path:

- fluid conservatives include `D_`, `s^C_`, `tau_`, magnetic field `b^C_`, and, when realistic EOS support is active, `Dye_` and `Dymu_`;
- fluid primitives include `T_eps_`, `rho_`, `u^C_`, and, when realistic EOS support is active, `ye_` and `ymu_`;
- metric variables include `alp_metric_`, `beta_metric^C_`, `Xvec_metric^C_`, and `psi_metric_`;
- optional GW-backreaction variables include `U_br_`, `h_00_`, and related quantities.

5.3 Radiation variables

When `M1` is compiled, each radiation species contributes:

- transported moments `nrad^KSP_`, `erad^KSP_`, `frad^KSP^C_`;
- microphysical source coefficients `kappa_a^KSP_`, `kappa_s^KSP_`, `Q_er^KSP_`, `kappa_nr^KSP_`, `Q_nr^KSP_`, and `tau_path^KSP_`.

The active five-spec build therefore adds five copies of the number-energy-flux block and five copies of the source-coefficient block.

5.4 Internal M1 state conventions

The internal M1 transport state is defined in `BHAC/src/m1/mod_m1_internal.t`:

$$\mathbf{U}_{\mathrm{rad}} = (E, F_1, F_2, F_3), \tag{1}$$

with the internal indexing

$$\texttt{m1_energy_} = 1, \qquad \texttt{m1_flux\^{}C_} = \texttt{m1_energy_} + C.$$

The closure helper structure in `mod_m1_closure.t` stores derived quantities such as:

- lab-frame moments `E`, `F_low`, `F_up`, `F2`;
- fluid velocity `vel` and Lorentz factor `W`;
- closure variables `zeta`, `chi`, `dthin`, `dthick`;
- fluid-frame moments `J`, `H_low`, `H_up`, `Gamma`.

6 Geometry, Coordinates, and Metric Infrastructure

6.1 Physics and numerical role

The geometry layer does three jobs:

1. define the background coordinate mapping and metric coefficients;
2. compute geometric operators such as gradients, divergences, curls, and reconstruction helpers;
3. provide the metric and source-term infrastructure needed by GRMHD, CFC, GW backreaction, and M1 transport.

6.2 Core geometry routines in `geometry.t`

Table 3: Routine inventory for `BHAC/src/geometry/geometry.t`.

Routine	Purpose	Main logic / variables
`set_pole`	pole handling	Sets geometry-dependent information near coordinate singularities.
`getgridgeo`	per-grid geometry allocation	Builds the geometry container for one grid.
`dealloc_myM`	release geometry	Frees per-grid metric/geometry storage.

Routine	Purpose	Main logic / variables
`putgridgeo`	store geometry	Writes geometry structures back into the grid container.
`fillgeo`	metric fill and geometric factors	Populates metric coefficients, volume factors, and related grid geometry.
`gradient`	gradient operator	Computes directional derivatives on the grid box.
`upwindGradientS`	upwind scalar gradient	Used in source terms and reconstruction-sensitive contexts.
`gradientS`	scalar gradient helper	Scalar-specific variant of `gradient`.
`divvector`	divergence of a vector	Computes $\nabla_i v^i$ with geometry factors.
`curl3`	three-dimensional curl	Computes the spatial curl for vector diagnostics.
`curlvector`	generic curl wrapper	Higher-level curl helper with direction control.
`divvectorS`	divergence helper	Scalar-form divergence interface.
`rec`	reconstruction helper	Builds left/right reconstructed interface states.

6.3 Metric interface in `mod_metric.t`

The core metric module provides local metric access independent of the specific coordinate family. The main visible routines are:

- `init_coord_check`: validate the chosen coordinate setup;
- `get_g4`: compute the covariant spacetime metric $g_{\mu\nu}$ and optionally its derivatives;
- `g4inv`: compute the inverse metric $g^{\mu\nu}$;
- `get_sqrtgamma`: compute $\sqrt{\gamma}$, the determinant of the spatial metric.

The actual coordinate systems are implemented in the family of `mod_coord_*.t` files. These define analytic or numerical coordinate maps such as Cartesian, spherical, Kerr-Schild-like, black-hole-adapted, cylindrical, and CFC coordinates.

6.4 CFC metric solver

The active CFC driver is `BHAC/src/geometry/cfc/mod_cfc.t`. The solver treats the spacetime in conformal-flatness form,

$$\gamma_{ij} = \psi^4 \hat{\gamma}_{ij}, \tag{2}$$

and evolves or solves for the metric potentials:

- the conformal factor ψ;
- the lapse α;
- the shift vector β^i.

The high-level routines are:

- `cfc_update_flag`: determine whether the metric should be updated at the current step;
- `cfc_solver_activate`: enable multigrid and register CFC settings;
- `cfc_metric_init`: initialize the CFC metric from imported or user-defined data;

- `cfc_solve_metric`: solve the elliptic CFC equations during evolution;
- `cfc_check_metric`: optional quality control on the metric fields.

The actual elliptic solves are split into specialized modules:

- `mod_cfc_psi.t` for the conformal factor;
- `mod_cfc_alp.t` for the lapse;
- `mod_cfc_beta.t` for the shift;
- `mod_cfc_parameters.t` for user-facing controls and scheduling.

6.5 GW backreaction

The GW backreaction driver lives in `BHAC/src/geometry/cfc/mod_gw_br.t`. Its public interface is intentionally small:

- `gw_br_activate`;
- `gw_br_update_flag`;
- `solve_gw_br`.

Its job is to compute post-Newtonian or quadrupole-style backreaction corrections using quantities such as:

- matter source combinations such as σ and enthalpy-weighted velocity sources;
- potentials `U_br_` and related derivatives;
- quadrupole-like tensors through modules such as `mod_gw_br_Iij_Qij.t`.

7 EOS Framework

7.1 Generic EOS interface in `mod_eos.t`

The generic EOS module is a pointer-based dispatch layer. It does not implement one EOS by itself; instead, it stores the active EOS type and a table of function pointers for pressure, energy, temperature, sound speed, composition-dependent helper calls, and atmosphere support.

The central design point is that all physics layers above the EOS call the generic interfaces:

- `eos_get_pressure_one_grid`;
- `eos_get_eps_one_grid`;
- `eos_get_cs2_one_grid`;
- `eos_get_temp_one_grid`;
- `eos_eps_get_all_one_grid`;
- `eos_temp_get_all_one_grid`;
- `eos_get_all_beta_eqm_one_grid`.

The helpers

`eos_uses_ye` and `eos_has_ymu`

are the key runtime queries that let the rest of the code distinguish:

- plain ideal-gas or polytropic EOS;
- 3D realistic table EOS depending on (ρ, T, Y_e);
- 4D leptonic EOS depending on (ρ, T, Y_e, Y_μ).

7.2 Atmosphere model

`mod_eos.t` also contains atmosphere support:

- `eos_atmo_activate`;
- `eos_initialize_atmo`;
- `atmo_get_pressure_one_grid`;
- `atmo_get_eps_one_grid`;
- `atmo_get_cs2_one_grid`.

The atmosphere can be:

- a simple polytropic/ideal-gas atmosphere;
- a beta-equilibrium atmosphere based on the active realistic EOS.

This atmosphere layer defines the lower bounds `small_rho`, `small_eps`, `small_press`, `small_temp`, `small_D`, and `small_tau` used by primitive recovery and small-value replacement.

7.3 Tabulated EOS in `mod_eos_tabulated.t`

The tabulated EOS module implements realistic 3D table access. The independent variables are density, temperature, and electron fraction:

$$(\rho, T, Y_e) \longrightarrow \left(p, \epsilon, c_s^2, s, \mu_e, \mu_p, \mu_n, \ldots\right). \tag{3}$$

The module supports at least two reader families:

- `scollapse`;
- `compose`.

The central routines are listed in Table 4.

Table 4: Routine inventory for `BHAC/src/eos_modules/mod_eos_tabulated.t`.

Routine	Purpose	Main logic / variables
`eos_tabulated_read_params`	select table reader	Switch between `scollapse` and `compose`.
`eos_tabulated_activate`	register tabulated EOS	Binds generic EOS pointers to tabulated implementations.
`tabulated_get_pressure_one_grid`	$p(\rho, \epsilon, Y_e)$ or $p(\rho, T, Y_e)$	Converts $\epsilon \to T$ when needed, then interpolates pressure.
`tabulated_get_eps_one_grid`	$\epsilon(\rho, T, Y_e)$	Direct interpolation of specific internal energy.
`tabulated_get_cs2_one_grid`	sound speed	Returns bounded relativistic c_s^2.
`tabulated_get_temp_one_grid`	invert $\epsilon \to T$	Root-finds or interpolates the temperature for fixed ρ and Y_e.
`tabulated_eps_get_all_one_grid`	bulk query at fixed ϵ	Returns thermodynamics and composition with optional chemical potentials.
`tabulated_temp_get_all_one_grid`	bulk query at fixed T	Direct table query used heavily by M1 and con2prim.

Routine	Purpose	Main logic / variables
`tabulated_get_eps_range`	EOS validity range	Returns the ϵ interval spanned by the table at given ρ and Y_e.
`tabulated_logeps_to_logtemp`	energy-temperature inversion	Root-finds T in log space using the table and EOS consistency.
`tabulated_get_all_beta_equilibrium`	beta-equilibrium EOS state	Solves for a beta-equilibrium composition at fixed (ρ, T).

Numerically, the tabulated EOS path relies on interpolation tables stored in `mod_eos_tabulated_parameters` and on interpolation kernels in `mod_eos_interpolation.t`. The key algorithmic feature is that many queries reduce to:

1. convert the physical inputs into table coordinates;
2. clamp or reject values outside the validity range;
3. interpolate one or several tables;
4. if only ϵ is known, invert for T first and then interpolate the desired quantity.

7.4 Leptonic 4D EOS in `mod_eos_leptonic.t`

The leptonic EOS extension adds a physically separate treatment of baryons, electrons/positrons, and muons/antimuons. The active architecture uses three tables:

1. a baryon table indexed by (ρ, T, Y_p) with $Y_p = Y_e + Y_\mu$;
2. an electronic leptonic table indexed by (ρ, T, Y_e);
3. a muonic leptonic table indexed by $(\rho, T, \log Y_\mu)$.

The total thermodynamics are assembled as

$$P_{\rm tot} = P_{\rm baryon} + P_e + P_\mu, \tag{4}$$
$$\epsilon_{\rm tot} = \epsilon_{\rm baryon} + \epsilon_e + \epsilon_\mu, \tag{5}$$
$$s_{\rm tot} = s_{\rm baryon} + s_e + s_\mu. \tag{6}$$

The main routines are listed in Table 5.

Table 5: Routine inventory for `BHAC/src/eos_modules/leptonic/mod_eos_leptonic.t`.

Routine	Purpose	Main logic / variables
`eos_leptonic_activate`	register 4D leptonic EOS	Reads baryon and leptonic tables and binds the generic EOS pointers.
`lep_bound_ye_ymu`	composition bounding	Clamps Y_e and Y_μ, builds $Y_p = Y_e + Y_\mu$, and optionally resets Y_μ if Y_p would exceed the baryon-table range.
`lep_bound_rho_temp`	thermodynamic bounding	Clamps density and temperature to the 4D table limits.
`leptonic_eval_total_from_logs`	total thermodynamic evaluator	Interpolates the baryon, electron, and muon tables and assembles the total state.

Routine	Purpose	Main logic / variables
`leptonic_get_pressure_one_grid`	pressure query	Uses either an input temperature or a prior inversion of $\epsilon \to T$.
`leptonic_get_eps_one_grid`	internal-energy query	Fixed-temperature query into the total EOS.
`leptonic_get_cs2_one_grid`	sound speed query	Returns total c_s^2 with physical clamping.
`leptonic_get_temp_one_grid`	invert $\epsilon \to T$	Root-finds the temperature against the assembled 4D EOS.
`leptonic_get_eps_range`	validity range	Returns the minimum and maximum ϵ available at fixed (ρ, Y_e, Y_μ).
`leptonic_temp_get_all_one_grid`	fixed-T query	Returns the assembled thermodynamics and chemical potentials.
`leptonic_eps_get_all_one_grid`	fixed-ϵ query	Calls the temperature inversion first, then the full evaluator.
`leptonic_find_ye_of_mumu`	invert Y_e from μ_μ	Root-find helper for muon equilibrium.
`leptonic_find_betaeq_ye`	beta-equilibrium Y_e	Root-find helper for beta-equilibrium proton content.
`leptonic_get_all_beta_eqm_one_grid`	beta-equilibrium state	Returns beta-equilibrium composition and optionally a consistent Y_μ.

The parameter module `mod_eos_leptonic_parameters.t` defines:

- the table names `baryon_table_name` and `leptonic_table_name`;
- the table axes and data arrays;
- bounds such as `lep_eos_rhomin`, `lep_eos_tempmax`, `lep_eos_ymumin`;
- behavioral flags such as `lep_use_muons`, `lep_add_ele_contribution`, `lep_fix_ymu_high_yp`, and `lep_extend_table_high`.

7.5 Why the generic EOS helpers matter

Several major code paths deliberately call `eos_uses_ye()` and `eos_has_ymu()` instead of checking a specific EOS type. This is essential because:

- both tabulated and leptonic EOSs require `Ye`;
- only the leptonic EOS requires `Ymu`;
- the rest of the hydro, import, and M1 source code should branch on *which primitive variables are physically active*, not on a hard-coded EOS integer.

8 GRMHD Physics Layer

8.1 Conserved and primitive variables

The GRMHD layer in `BHAC/src/imhd/amrvacphys.t` evolves the usual conservative state. In the active relativistic primitive formulation, the code uses

$$D = \rho W, \tag{7}$$

$$\xi = \rho h W^2, \tag{8}$$

$$S_i = (\xi + B^2) v_i - (v \cdot B) B_i, \tag{9}$$

$$\tau = \xi - p - D + \frac{1}{2}\left(B^2 + B^2 v^2 - (v \cdot B)^2\right), \tag{10}$$

with additional composition conservatives

$$DY_e \equiv \texttt{Dye_}, \qquad DY_\mu \equiv \texttt{Dymu_}. \tag{11}$$

Here W is the fluid Lorentz factor, $h = 1+\epsilon+p/\rho$ is the specific enthalpy, and B^i is the magnetic field in the Eulerian frame.

8.2 Interface routines in `amrvacphys.t`

Table 6: Main GRMHD interface routines in `BHAC/src/imhd/amrvacphys.t`.

Routine	Purpose	Main logic / variables
`getdt`	local time-step contribution	Provides the GRMHD contribution to the CFL step.
`checkglobaldata`	validate global physics settings	Initializes or checks floors and equation-of-state consistency.
`initglobaldata`	initialize physics parameters	Sets default γ, adiabatic constants, and related global parameters.
`conserve`	primitive-to-conservative wrapper	Calls the active `conserven` implementation.
`conserven`	primitive-to-conservative transform	There are dynamic-spacetime and static-spacetime branches; both build `D_`, `s^C_`, `tau_`, composition conservatives, and preserve magnetic fields.
`primitive`	conservative-to-primitive wrapper	Calls the dedicated con2prim module.
`getv`	velocity extraction	Returns transport velocities for a given direction.
`get_cbounds`	characteristic bounds	Builds left/right wave-speed bounds for the Riemann solver.
`get_lambda`	characteristic speeds	Includes M1 contributions when radiation transport is active.
`getcmax` / `getcmax_old`	scalar max-speed estimates	Used by CFL logic and older flux paths.
`getflux`	flux calculation	Builds GRMHD fluxes and switches off transport for variables that should not be advected directly.

Routine	Purpose	Main logic / variables
`addgeometry`	geometric source terms	Applies metric-derivative source terms and couples to M1 geometry when active.
`addsource`	user/source-driver wrapper	Routes source splitting to physics and user hooks.
`getcurrent`	current diagnostic	Computes spatial current from the magnetic field.
`addsource_elecheating`	heating source helper	Specialized electron-entropy or heating source path.
`calheatingrate`	heating-rate diagnostic	Computes heating-related derived quantities.
`e_to_rhos` / `rhos_to_e`	variable conversion helper	Converts between energy and entropy-like storage variants.
`implicit_update`	stiff source update	Calls the implicit update path used for radiation-coupled source terms.

8.3 Primitive recovery in `mod_imhd_con2prim.t`

The routine `imhd_con2prim` is the active conservative-to-primitive recovery for the realistic EOS path. The method is a root solve in auxiliary variables denoted by μ_+ and μ in the code, together with repeated EOS calls to recover:

$$\rho, \quad \epsilon, \quad T, \quad Y_e, \quad Y_\mu, \quad p, \quad W, \quad v^i.$$

The internal workflow is:

1. extract and rescale the conservative state;
2. determine composition from `Dye_/D_` and `Dymu_/D_` when active;
3. compute bounds for the auxiliary unknown;
4. solve the root problem using constrained Newton-Raphson and Brent fallback;
5. reconstruct the primitive state and limit it against floors and Lorentz-factor bounds;
6. call the EOS to obtain consistent thermodynamics.

The module-specific routines are:

- `imhd_con2prim_setup`;
- `imhd_con2prim`;
- internal helper `get_vars`;
- scalar root functions `func_for_mu_plus`, `d_mu_plus`, and `func_for_mu`.

8.4 Intermediate variables in `mod_imhd_intermediate.t`

This module gathers all derived hydrodynamic and electrodynamic quantities needed outside the raw primitive/conservative transforms. The most important routine is

`imhd_get_intermediate_variables`,

which can return any subset of:

- γ_{ij} and γ^{ij};
- W, W^2, and v^2;
- $B \cdot v$, b^μ, b^2, and Eulerian electric field quantities;
- thermal and total enthalpy;
- thermal and total pressure;
- Bernoulli-based unboundness criterion.

Other important routines in this file are:

- `dysp_get_lfac`;
- `imhd_get_Efield_Eulerian`;
- `imhd_handle_small_values`;
- `Eos_update_one_grid` and `Eos_update_one_point`;
- `imhd_modify_wLR`;
- `imhd_get_tilde_U`, `imhd_get_tilde_S_i`, `imhd_get_tilde_S`;
- `conformal_transformation` and `inverse_conformal_trans`;
- `imhd_ppm_flatcd` and `imhd_ppm_flatsh`.

8.5 Auxiliary GRMHD diagnostics in `mod_physaux.t`

The diagnostics and stress-energy helper layer is implemented in `BHAC/src/imhd/mod_physaux.t`. Its main routines are:

- `get_b2` and `get_b2_pt`;
- `get_u4`;
- `get_b4`;
- `get_TEM4_ud`;
- `get_TPAKE4_ud`;
- `get_TEN4_ud`.

These routines are used by diagnostics, post-processing, outflow calculations, and any module that needs explicitly geometric stress-energy or four-vector quantities rather than just primitive variables.

9 M1 Radiation Transport

9.1 Physical model

The active M1 implementation evolves number, energy, and flux moments for each radiation species:

$$\partial_t(\sqrt{\gamma}\, N) + \partial_i \mathcal{F}^i_N = \sqrt{\gamma}\, G_N, \tag{12}$$

$$\partial_t(\sqrt{\gamma}\, E) + \partial_i \mathcal{F}^i_E = \sqrt{\gamma}\, G_E, \tag{13}$$

$$\partial_t(\sqrt{\gamma}\, F_j) + \partial_i \mathcal{F}^i_{F_j} = \sqrt{\gamma}\, G_{F_j}, \tag{14}$$

where the source terms contain:

- emissivities;
- absorption opacities;
- scattering opacities;
- number-emission and number-absorption terms;
- geometric source terms from the curved metric.

The moment system is closed through a Minerbo or Levermore closure by introducing an Eddington factor χ and blending optically thin and thick limits:

$$P^{ij} = d_{\text{thick}} P^{ij}_{\text{thick}} + d_{\text{thin}} P^{ij}_{\text{thin}}, \qquad d_{\text{thin}} = \frac{3\chi - 1}{2}, \qquad d_{\text{thick}} = 1 - d_{\text{thin}}. \tag{15}$$

9.2 Species bookkeeping and startup

The species-selection logic is in `BHAC/src/m1/mod_m1.t`. The routine `m1_startup`:

- resets species indices;
- assigns active indices for ν_e, $\bar{\nu}_e$, ν_x, ν_μ, $\bar{\nu}_\mu$, and optionally photons;
- activates the closure module;
- activates gray M1 microphysics;
- reads Weakhub tables when the Weakhub path is selected.

The user-facing species and rate-mode parameters live in `BHAC/src/m1/mod_m1_eas_param.t`. The most important ones are:

- `m1_i_nue`, `m1_i_nuebar`, `m1_i_nux`, `m1_i_mu`, `m1_i_mubar`;
- `m1_rates_Weakhub`, `m1_rates_analytical`;
- `beta_decay`, `plasmon_decay`, `bremstrahlung`, `pair_annihil`;
- the fluid-variable indexing `idx_rho`, `idx_T`, `idx_Ye`, `idx_Ymu`.

9.3 Transport routines in `mod_m1.t`

Table 7: Routine inventory for `BHAC/src/m1/mod_m1.t`.

Routine	Purpose	Main logic / variables
`m1_startup`	initialize M1 subsystem	Set species indices, closure, and rate backends.
`m1_read_Weakhub`	table loading wrapper	Read the active Weakhub table through `mod_Weakhub_reader.t`.
`m1_get_wavespeeds`	characteristic speeds	Blend optically thin and thick wave speeds using closure variables.
`m1_get_fluxes`	transport fluxes	Build number, energy, and momentum fluxes in curved spacetime.
`m1_correct_asymptotic_fluxes_old`	historical corrector	Kept as an older implementation.
`m1_correct_asymptotic_fluxes`	asymptotic flux correction	Repairs interface fluxes in optically thin/thick asymptotic regimes.

Routine	Purpose	Main logic / variables
`m1_add_geometrical_sources`	metric source terms	Adds lapse, shift, and spatial-metric derivative sources.
`m1_add_collisional_sources`	stiff microphysical coupling	Calls the implicit collisional update and backreaction logic.

9.4 Closure and realizability in `mod_m1_closure.t`

The closure layer is the core admissibility and frame-conversion engine. The most important routines are:

- `m1_closure_type_activate`;
- `m1_closure_activate`;
- `m1_update_closure`;
- `m1_enforce_realizability`;
- `m1_update_closure_ixD`.

The pointwise closure update `m1_update_closure_ixD` does the heavy lifting:

1. compute metric-raised and metric-lowered fluxes;
2. evaluate the closure parameter ζ by root finding if requested;
3. compute χ, J, H_i, and the normalization factor `Gamma`;
4. limit non-finite or acausal states;
5. if necessary rescale F_i so that
$$F_i F^i < E^2 \tag{16}$$
holds after the update.

The separate routine `m1_enforce_realizability` is applied on the densitized transported state and enforces:
$$E \geq E_{\mathrm{atmo}}, \qquad N \geq N_{\mathrm{atmo}}, \qquad F_i F^i < E^2. \tag{17}$$

This routine is now a key safety barrier against negative energies, negative number densities, NaNs, and the previously observed $E^2 < F^2$ failure mode.

9.5 Implicit collisional source solve in `mod_m1_collisional.t`

The collisional update is formulated as a backward-Euler solve for the stiff local source operator:
$$\mathbf{U}^{n+1} = \mathbf{U}^{*} + \Delta t\, \mathbf{G}\left(\mathbf{U}^{n+1}\right). \tag{18}$$

The central entry point is `m1_get_implicit_collisional_sources`. Its workflow is:

1. remove densitization by dividing by `sqrtg`;
2. update the closure so that J, H_i, and Γ are consistent with the current iterate;
3. store the explicit state and microphysical coefficients;
4. solve the nonlinear system with a multidimensional Newton method and fallback multistart logic;

5. enforce post-solve admissibility of the radiation state;
6. update number and lepton-number source terms through `m1_get_implicit_n_source`;
7. redensitize the updated state.

The module also contains:

- Jacobian builders `m1_collisional_jacobian`, `m1_sources_jacobian`, and linearized variants;
- source evaluators `m1_collisional_sources`, `m1_sources_func`, and linearized variants;
- `get_iter_initguess`;
- unit-test helpers for isolated local solves.

9.6 Rates and emissivity-absorption-scattering infrastructure in `mod_m1_eas.t`

The file `BHAC/src/m1/mod_m1_eas.t` is the main microphysical dispatcher. It handles:

- activation of the gray microphysics backend;
- computation of per-cell emissivity, absorption, scattering, number-emission, and number-absorption rates;
- optional beta-equilibrium relaxation of the fluid state;
- both Weakhub and analytical three-spec source models.

Table 8: Routine inventory for `BHAC/src/m1/mod_m1_eas.t`.

Routine	Purpose	Main logic / variables
`m1_eas_gray_activate`	choose gray backend	Registers the active gray rate provider, currently Weakhub for tabulated gray rates.
`m1_eas_activate`	consistency check	Ensures a rate backend is selected.
`m1_allocate_eas_ixD` / `m1_deallocate_eas_ixD`	pointwise storage management	Allocate per-cell per-species source-coefficient buffers.
`m1_add_eas`	cell-wise EAS assembly	For each species, compute closure variables, call the active rate model, and write `wradimpl`.
`calculate_m1_eas_ixD`	main local rate calculator	Dispatch between Weakhub and analytical rates, apply beta-equilibrium coupling, and return `eas`.
`m1_eas_analytic_brems`	heavy-lepton bremsstrahlung	Add bremsstrahlung emission and number source terms.
`m1_eas_analytic_pair`	pair annihilation	Add pair-process contributions.
`m1_eas_analytic_plasmon`	plasmon decay	Add plasmon-decay contributions.
`m1_eas_analytic_kappa_AN`	analytical opacities	Compute analytical absorption and number-opacity pieces.
`Q_Ruffert`	Ruffert-like free emission	Compute free emissivities before optical-depth reduction.

Routine	Purpose	Main logic / variables
`m1_eas_analytic`	analytical 3-spec model	Assemble the full analytical source vector for the active species.

9.7 Weakhub gray tables

The active Weakhub gray table reader is `BHAC/src/m1/mod_Weakhub_reader.t`. It reads HDF5 tables containing:

- energy absorption opacities;
- number absorption opacities;
- scattering opacities;
- 3D or 4D table axes in $(\log\rho, \log T, Y_e)$ or $(\log\rho, \log T, Y_e, \log Y_\mu)$;
- species count and table metadata.

For gray transport, `mod_m1_eas_microphysical_gray.t` performs:

- trilinear interpolation for three-spec mode;
- quadrilinear interpolation for four-dimensional muonic mode;
- per-species lookup of `k_a`, `k_s`, and `k_n`.

This is the reason the active five-spec muonic branch requires a true 4D muonic Weakhub table.

9.8 Analytical rates

The analytical path is currently restricted to three species. Its physics content mirrors the FIL-style three-spec treatment:

- charged-current beta processes for ν_e and $\bar{\nu}_e$;
- neutral-current scattering;
- optional pair annihilation;
- optional nucleon bremsstrahlung;
- optional plasmon decay.

The active code therefore supports:

- **three species:** either Weakhub or analytical rates;
- **five species:** Weakhub only.

9.9 Backreaction, beta equilibrium, diagnostics, and optical depth

Backreaction. The routine `m1_add_backreaction` in `mod_m1_backreaction.t` transfers collisional source terms from the radiation subsystem back into the hydrodynamic conserved variables. It updates:

- `tau_` through the summed radiation energy source;
- `Dye_` through ν_e and $\bar{\nu}_e$ number sources;
- `Dymu_` through ν_μ and $\bar{\nu}_\mu$ number sources when the leptonic EOS is active.

Beta equilibrium. The file `mod_m1_beta_eq.t` solves a two-variable equilibrium problem for (Y_e, T) at fixed density and fixed Y_μ. The two residual equations are:

1. lepton-number conservation between matter and trapped neutrinos;
2. total internal-energy consistency between matter and trapped neutrino moments.

The main entry point is `m1_get_beta_equilibrium`, backed by:

- `m1_beta_equilib_function`;
- `m1_beta_equilib_jacobian`;
- `m1_beta_equilib_min`;
- `betaeq_rootfinding_bounded_newton_raphson`.

Diagnostics. The file `mod_m1_diagnostics.t` computes derived neutrino chemical-potential and number-density diagnostics through:

- `m1_compute_diagnostics`;
- `m1_diagnostics_mu`.

Optical depth. The optical-depth module `mod_m1_tau_optical.t` contains `m1_add_tau`. It:

1. computes equilibrium opacities using either Weakhub or the analytical rate path;
2. estimates an empirical seed optical depth;
3. sweeps neighboring cells and computes a metric-weighted path integral:

$$\tau_{\text{path}} \approx \bar{\kappa}\, ds + \tau_{\text{empirical}}. \tag{19}$$

10 I/O, Conversion, Import, and Healpix Detectors

10.1 Runtime I/O in `amrio.t`

The main runtime I/O hub is `BHAC/src/amrvacio/amrio.t`. It handles:

- command-line parsing through `readcommandline`;
- full namelist parsing and runtime validation through `readparameters`;
- snapshot write/read in parallel and non-parallel formats;
- HDF5 snapshot I/O;
- XDMF metadata generation for HDF5 outputs;
- default logging through `printlog_default`.

The main routines are:

- `readcommandline`, `readparameters`;
- `saveamrfile`;
- `write_snapshot`, `write_snapshot_nopar`, `write_snapshot_noparf`, `write_snapshot_hdf5`;
- `read_snapshot`, `read_snapshot_nopar`, `read_snapshot_noparf`, `read_snapshot_hdf5`;
- `MakeXDMF`;
- `printlog_default`.

10.2 Conversion and post-processing in `convert.t`

The conversion layer `BHAC/src/amrvacio/convert.t` is writer-heavy. Its job is to transform in-memory AMR data into plot, slice, and unstructured output formats. The routine groups are:

- orchestration: `generate_plotfile`, `oneblock`, `onegrid`, `calc_grid`;
- header/metadata: `getheadernames`, `calc_x`;
- IDL/DX/VTK/VTU/VTI/Tecplot writers: `valout_idl`, `valout_dx`, `write_vtk`, `write_vti`, `write_pvtu`, `tecplot`, `tecplot_mpi`, `unstructuredvtk`, `unstructuredvtk_mpi`, and related helper routines;
- radiation-aware post-processing: `post_rad`.

10.3 Analysis hooks

The file `BHAC/src/amrvacio/analysis.t` contains the example routine `write_analysis`. It is intentionally a template-style extension point for user-defined integrated diagnostics.

10.4 External initial-data import

The import layer has three main parts:

- `mod_c2b.t`: 2D C2B profile import;
- `mod_xns.t`: XNS profile import;
- `mod_c2b_cart3d.t`: FIL/Carpet-style 3D HDF5 import.

`mod_c2b.t`. This module manages structured 2D initial-data profiles and provides:

- allocation and destruction;
- dimension and grid readers;
- metric and primitive/conservative profile readers;
- 1D and 2D interpolation helpers.

`mod_xns.t`. This module provides the same role for XNS data with routines for allocation, file reading, grid extraction, profile extraction, and 1D/2D interpolation.

`mod_c2b_cart3d.t`. This is the key module for 3D import from FIL/ETK/Carpet-like HDF5 output. Its responsibilities are:

1. represent each imported variable as a hierarchy of refinement levels and Carpet data blocks;
2. read the HDF5 attributes that define origin, spacing, ghost zones, and valid interpolation range;
3. find which refinement level and block contain a requested point;
4. interpolate one-point values using linear, Hermite, or higher-order Lagrangian interpolation.

The key routines are:

- `init_single_variable`, `init_single_reflevel`, `init_single_datablock`;
- `load_selected_ref_and_datablock`;
- `destroy_single_variable`, `destroy_single_reflevel`;

- hdf5_read_attr_single_datablock, hdf5_read_single_dataset;
- find_point_in_which_level, find_point_in_which_datablock, find_point;
- interp_cart3d_carpet_variable_one_point, interp_one_point_from_dataset;
- interpolation kernels linear_interp_3d, Hermitian_interp_3order_3d, lagrangian_interp_high_order_

10.5 Healpix outflow detectors

The detector module BHAC/src/amrvacio/mod_healpix_det.t computes shell and surface-integrated fluxes on spherical Healpix detectors. It supports:

- mass flux;
- unbound mass flux;
- Poynting flux;
- shell quantities such as radial velocity, asymptotic velocity, angular velocity, inverse plasma beta, and specific angular momentum;
- M1 radiation outflow per species.

The workflow is:

1. initialize detectors and parse the selected outflow names through initialize_healpix_detectors;
2. map Healpix indices to coordinates through get_coord_from_healpix_index;
3. for each detector pixel, interpolate the requested fluid or radiation quantity and project it onto the radial normal;
4. integrate over the sphere through surface_integral_outflow;
5. optionally dump detector snapshots to HDF5 through save_outflow_snap_to_hdf5.

Important routines include:

- initialize_healpix_detectors, destroy_healpix_detectors;
- initialize_single_detector, cal_flux_single_detector;
- calculate_flux_m1, calculate_flux_poynting, calculate_flux_mass, calculate_flux_unbound_mass;
- calculate_shell_inverse_beta, calculate_shell_omega_xy, calculate_shell_velo_parker_criteric calculate_shell_velo_asymptotic, calculate_shell_velo_r, calculate_shell_specific_j;
- projection_flux_on_sphere, surface_integral_outflow, calculate_all_outflow, save_outflow_snap

11 Tests, Example Workflows, and Practical Use

11.1 BNS test setups

The two key BNS setups in this workspace are:

- BHAC/tests/BNS_SFHO: baryonic SFHo EOS workflow;
- BHAC/tests/BNS_SFHO_muons: leptonic SFHo EOS with five M1 species.

11.2 What the muonic test is demonstrating

The muonic test is a reference setup for the full BHAC+ extension chain:

1. 3D import of fluid, metric, and radiation fields from FIL/Carpet HDF5 output;
2. leptonic EOS activation using a matching baryon table and 4D leptonic table;
3. five-spec radiation transport with electron and muon neutrinos;
4. Weakhub-based gray rates in 4D (ρ, T, Y_e, Y_μ);
5. Healpix outflow diagnostics for both fluid and radiation species.

11.3 Why imported variable ordering matters

The import path in `amrvacusr.t` must match the `setup_variables` layout exactly. This is especially important when:

- realistic EOS composition variables are compiled;
- the build uses three species versus five species;
- extra metric or GW variables are enabled.

For that reason, the `primnames` and `wnames` strings in the test parameter files are not cosmetic; they must reflect the actual order of the registered variables.

12 Extension Guide and Common Pitfalls

12.1 Adding a new EOS or modifying an existing one

The clean extension pattern is:

1. add a dedicated module in `BHAC/src/eos_modules`;
2. add any reader and parameter modules it needs;
3. bind the generic EOS pointers in `<new>_activate`;
4. use `eos_uses_ye()` and `eos_has_ymu()` in calling code instead of hard-coded EOS IDs whenever the real question is "which composition variables are active?".

12.2 Adding new radiation species or rate models

The key places to modify are:

- `mod_m1_eas_param.t` for species indices and runtime flags;
- `mod_m1.t` for startup indexing and transport assumptions;
- `mod_variables.t` for state-vector registration;
- `mod_m1_eas.t` for the actual rate logic;
- `mod_m1_backreaction.t` for matter-coupling consistency;
- test `amrvacusr.t` and `import.par` files for import and diagnostics.

12.3 Common failure modes

Radiation realizability. If the code produces NaNs, negative `erad`, negative `nrad`, or $E^2 < F^2$ violations, the first places to inspect are:

- `m1_enforce_realizability`;
- `m1_update_closure_ixD`;
- the post-update repairs in `finite_volume.t` and `finite_difference.t`;
- import routines that densitize radiation variables before the closure has repaired them.

Leptonic consistency. For leptonic EOS runs, verify all of the following at once:

- build with `TABEOS`;
- runtime `eos_type_input='leptonic'`;
- both `baryon_table_name` and `leptonic_table_name` are set;
- `lep_use_muons=.true.` if muon transport is enabled;
- imported data carry `ymu` and the muonic radiation moments.

Species-count mismatch. Five-spec logic is not a pure runtime switch. The build must also be compiled with `-nspecies=5`. Otherwise the source indices and imported state ordering do not match the compiled variable layout.

13 Utility Modules and Support Infrastructure

The generic module layer in `BHAC/src/modules` is not physics-specific, but it is essential for understanding the implementation:

- `mod_rootfinding.t`: one-dimensional and multidimensional root solves used by con2prim, EOS inversion, M1 closure, and collisional sources;
- `mod_interpolate.t`: trilinear and quadrilinear interpolation used by the Weakhub gray path;
- `mod_lu.t` and `mod_lnsrch.t`: dense linear algebra and line search helpers;
- `mod_constants.t`: physical constants and unit conversions used across several physics modules;
- `mod_timing.t`: accumulated runtime timers;
- `mod_oneblock.t`: utilities for block-wise operations;
- `mod_ngmres.t`, `mod_odeint.t`, `mod_rtsafe2D.t`, and `mod_random.t`: reusable numerics.

14 Summary

The active BHAC+ code in this workspace is best understood as a layered system:

1. a generic AMR and time-integration framework;
2. a GRMHD physics layer that owns conservative and primitive hydrodynamic variables;
3. a geometry and metric layer that provides curved-space operators and CFC/GW support;
4. an EOS framework that now supports both 3D baryonic and 4D leptonic thermodynamics;

5. an M1 transport layer that supports three-spec and five-spec radiation with Weakhub gray tables and a three-spec analytical alternative;
6. a robust import, conversion, and detector infrastructure that makes the code usable for realistic BNS workflows and direct comparison with FIL data products.

For development, the most important practical rule is to extend the code *through the active interfaces*:

- the generic EOS pointers in `mod_eos.t`;
- the variable registry in `mod_variables.t`;
- the M1 startup and source dispatch in `mod_m1.t` and `mod_m1_eas.t`;
- the import ordering in test `amrvacusr.t` files;
- the consistency guards in `amrio.t`.

This keeps new physics aligned with the actual compiled BHAC+ workflow instead of only working in isolated draft files.